

VLM-Guided Robot Reaching

Language-conditioned tabletop manipulation with a vision-language model in the loop

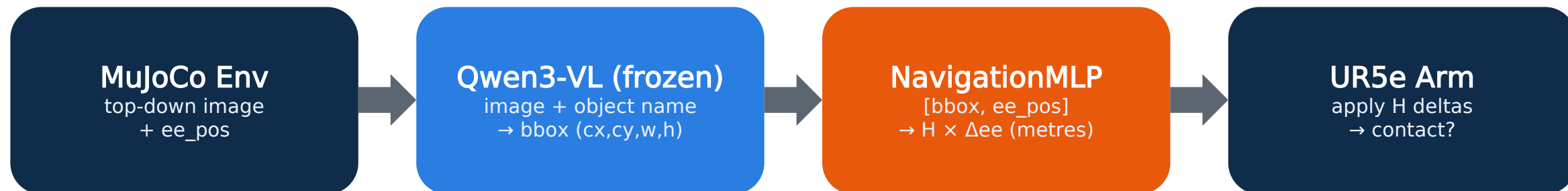
A frozen Qwen3-VL detector feeds a behaviour-cloned navigation policy that drives a UR5e arm to touch a named object.

Onuralp Güvercin · Deep Learning in Robotics · Final Project

Objective

- ▶ **Task:** a robot arm must reach out and touch a specific object named in natural language - e.g. "the red cube".
 - Scene: a tabletop with 2–4 differently coloured boxes and spheres at random positions (6 colours × 2 shapes).
 - The robot sees only a top-down camera image; it is never told the target's coordinates.
- ▶ **The central question**
 - Can a vision-language model act as the "eyes" that localise the target, while a small learned policy acts as the "hands" that move toward it?
- ▶ **Success metric**
 - An episode succeeds if the end-effector makes contact with - or comes within 5 cm of - the correct target.
 - Success rate, final EE-to-object distance, and steps taken are reported.

System Architecture



Closed loop: the policy executes H steps, then re-queries the VLM (H steps = 1 VLM call).

- Perception (VLM): localises the named object as a normalised bounding box - off-the-shelf, weights frozen.
- Control (MLP): 7-D input [bbox(4) + ee_pos(3)] → $3 \cdot H$ deltas, predicting H steps of motion in one shot. Trained purely by behaviour cloning - no RL, no VLM training.
- Horizon H is the same knob in both training and inference.
- Training: each sample regresses H consecutive expert deltas, so the MLP's output head is sized $3 \cdot H$.
- Inference: those H deltas run before the next VLM call - small H = more re-planning & VLM cost, large H = cheaper but more open-loop drift ($H = 1-5$ is swept).

Data Collection

- ▶ An automated scripted expert generates every demonstration - no human teleop, no learning here.
 - Its only job is to produce the correct answer the policy will later imitate; the intelligence lives in the data, not the controller.
- ▶ For each random scene (2-4 objects, rejection-sampled ≥ 10 cm apart, unique colour + shape)
 - every object in the scene is targeted in turn;
 - a straight-line controller drives the end-effector toward the target's (x, y) in 4 cm steps - capped so it never overshoots.
- ▶ Logged at every step
 - top-down RGB image (128×128) and end-effector position ee_pos (xyz);
 - the ground-truth bbox - the object's true 3-D position projected through the camera matrix (not a vision model).
- ▶ What gets saved
 - a reach ends on gripper-target contact (MuJoCo geom collision); only successful reaches are stored - one .pt file per trajectory.
 - parallelised across seeded worker processes → ~2,700 demonstrations.
- ▶ Action labels (Δee) are not stored - they are derived in training as $ee[t+1] - ee[t]$. The VLM is never used during collection.

Policy & Behaviour Cloning

► NavigationMLP - a residual MLP (width 512, 4 residual blocks, SiLU + LayerNorm).

- Input (7-D): $\text{bbox}(cx, cy, w, h)$ + normalised $\text{ee_pos}(x, y, z)$.
- Output (3·H): end-effector deltas Δ_{ee} in metres, H steps ahead.

► Trained by behaviour cloning (supervised regression)

- Target = the expert's next H step-to-step Δ_{ee} (the same horizon used at inference); loss = MSE on the deltas.
- Each training sample regresses a whole H-step block at once, so the policy learns to plan the horizon, not just one step.
- AdamW + warmup→cosine LR; the output head is zero-initialised so the policy starts from gentle, near-zero motions.

► Honest evaluation

- Stratified train/val/test split by colour+shape; samples from one trajectory never leak across splits.
- Test RMSE ≈ 0.0007 m - the policy reproduces expert deltas almost exactly.

Critical Design Choices

► Fixed bounding box per episode.

- The bbox is read once at the start - when the object is clearly visible - and held constant; the loop then closes on ee_pos, not on a re-detected box.
- As the arm moves over the target, the gripper occludes the object, so a fresh detection mid-reach would be unreliable or fail; holding the initial box avoids this.

► Delta actions with clamping (± 5 cm/step).

- The policy predicts relative motion (Δee), not an absolute target - every command is a small step from the current pose, not a jump to a far point.
- Each delta is clipped to ± 5 cm per axis ($\approx$ the expert's 4 cm step), so even a wrong prediction can only nudge the arm - keeping the closed loop stable.

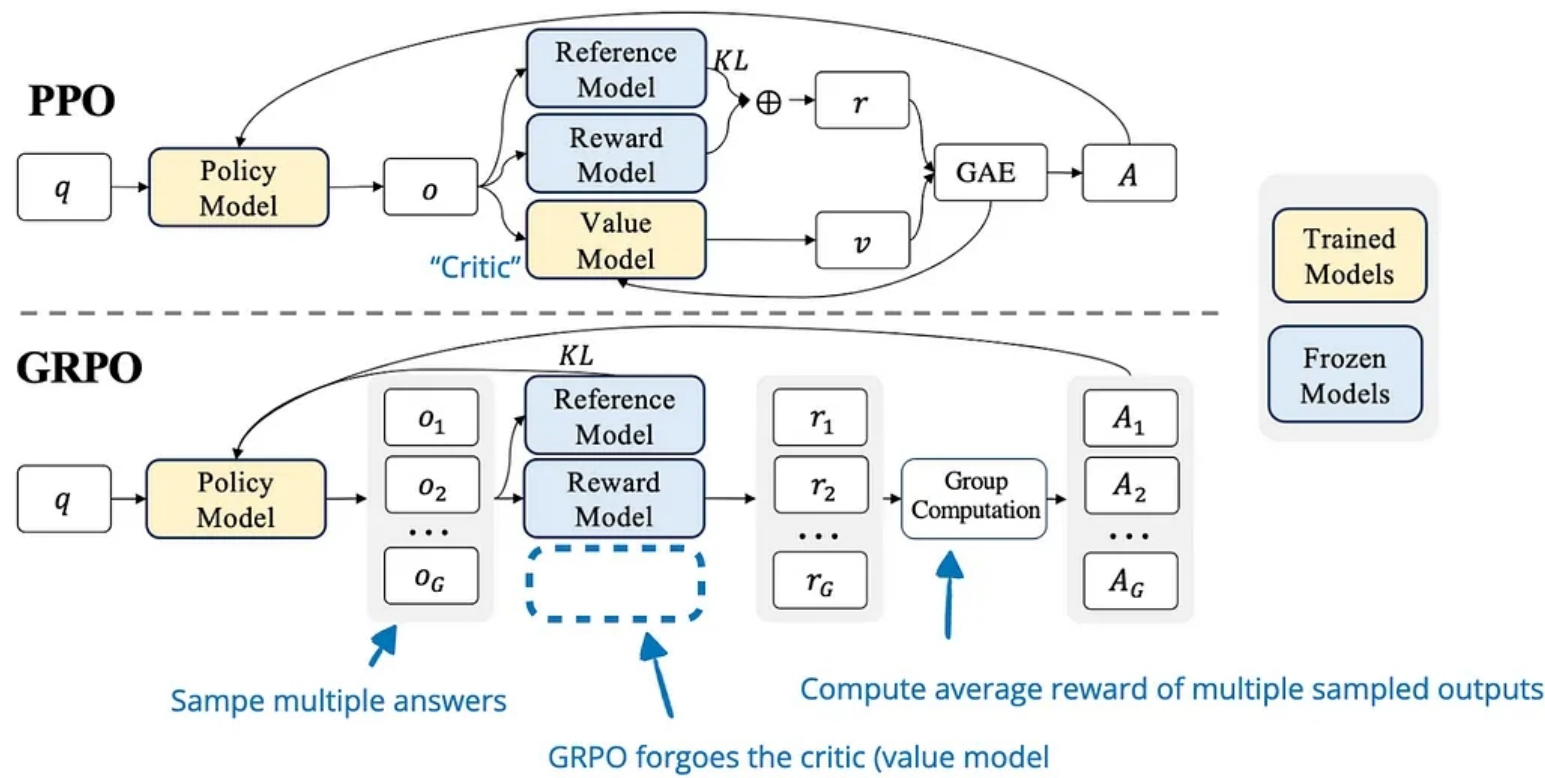
► Configurable horizon H (1-5).

- H sets how many predicted steps are executed before the VLM is queried again - directly trading perception cost against open-loop accuracy.
- Small H = frequent re-planning (more VLM calls, tighter tracking); large H = cheaper but more drift between detections.

The VLM Was Not Trained

- ▶ In this project Qwen3-VL-4B is used frozen, off-the-shelf, as the detector.
 - Fine-tuning a 4-billion-parameter vision-language model needs multi-GPU memory and long training runs - compute that was not available.
 - So the contribution is the end-to-end pipeline + the control policy, with the VLM as a plug-in perception module.
- ▶ **If compute had been available**
 - The VLM would be fine-tuned with GRPO (Group Relative Policy Optimization).
 - Reward = how well the predicted bbox matches the target (IoU / centre error), or even downstream reaching success.
 - This adapts the detector to the exact camera, lighting, and object set - directly attacking the VLM-vs-GT gap shown in the results.

PPO vs GRPO - The Idea



► PPO (top)

- needs a separate Value Model to estimate the baseline (advantage via GAE).
- = a second large network to train and store.

► GRPO (bottom)

- samples a GROUP of G answers per prompt,
- scores each with the reward model,
- uses the group's mean/std as the baseline - no critic at all.

PPO trains an extra Value Model ("critic"); GRPO replaces it by sampling a group and normalising rewards.
(DeepSeekMath, 2024)
VLM-Guided Robot Reaching

PPO vs GRPO - The Loss Functions

Shared importance ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

PPO

$$J_{PPO}(\theta) = E[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t)] - \beta \text{KL}[\pi_{\theta} \parallel \pi_{ref}]$$

$$\hat{A}_t = \text{GAE}(\delta_t, V_{\psi}(s_t))$$

- Advantage $\hat{A}$ comes from a learned critic V (GAE).
- **Extra value network → more memory & compute.**
- KL penalty keeps the policy near the reference.

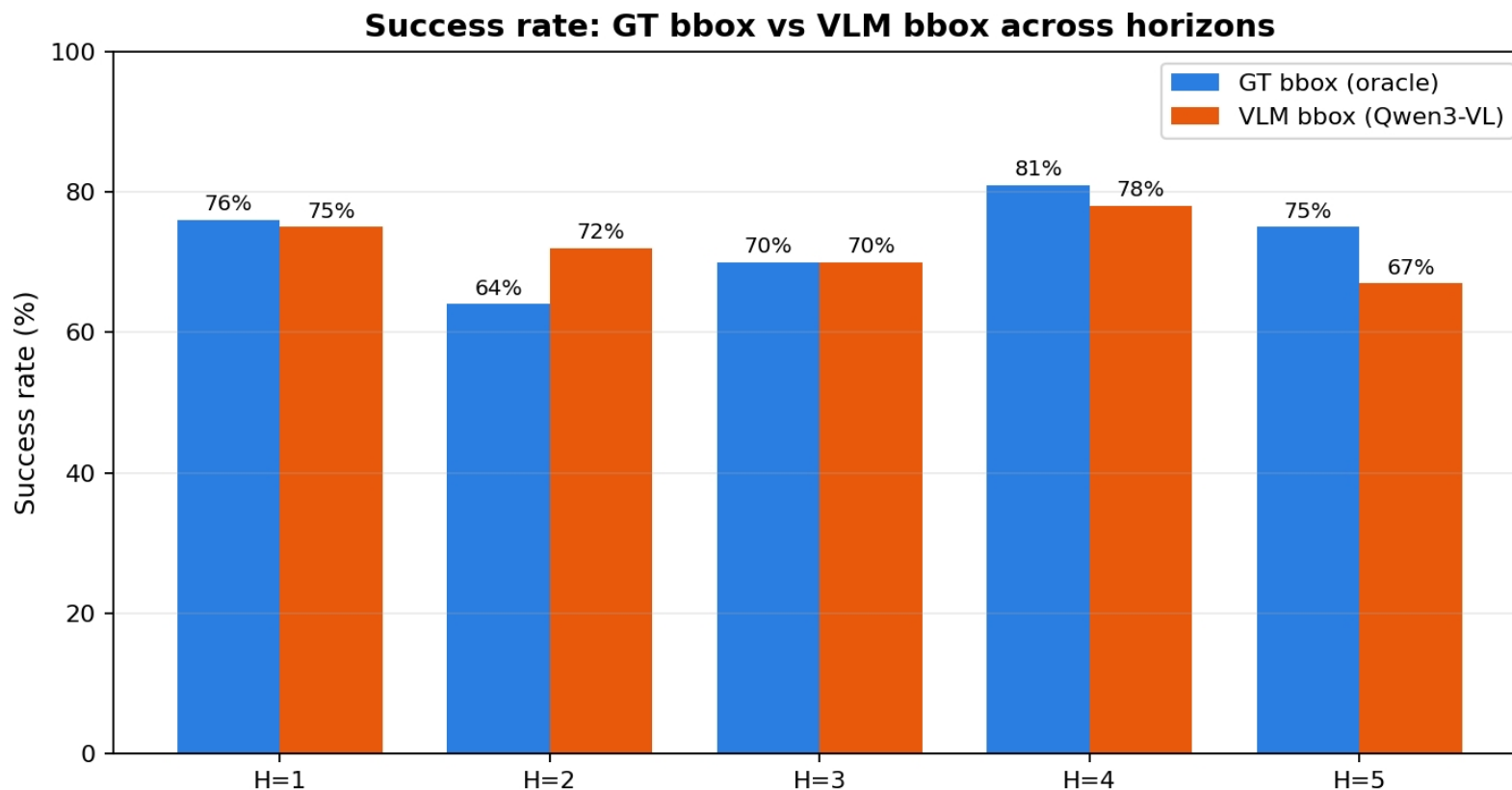
GRPO

$$J_{GRPO}(\theta) = E\left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i)\right] - \beta \text{KL}[\pi_{\theta} \parallel \pi_{ref}]$$

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}$$

- $\hat{A}$ = reward normalised within the group (no critic).
- **Group mean is the baseline → lighter & simpler.**
- Same clipped surrogate + KL to the reference.

Results · Success Rate



► Same policy in both runs - only the bbox source changes.

► **GT bbox: 64–81% across horizons.**

- Best at H=4 (81%); no clear monotonic trend - open-loop drift is not the dominant factor at this scale.

► **VLM bbox: 67–78%, within ≤ 9 pp of GT.**

- Qwen3-VL bboxes are competitive; the gap closes significantly vs prior expectations.

- H=5 shows the clearest VLM penalty (75% vs 67%).